

## Short Questions 3

Josh Manchester  
Department of Computer Science  
University of Colorado Colorado Springs  
Colorado Springs, CO 80918  
jmanches@uccs.edu

### Abstract

This paper covers three topics in Monte Carlo reinforcement learning: what the term “Monte Carlo” means, the Bellman equation and why it matters, and how Exploring Starts algorithms work along with their limitations. Monte Carlo methods learn value functions by running full episodes and averaging the returns without needing a model of the environment. The Bellman equation gives us the recursive relationship behind all value-based RL. Exploring Starts algorithms guarantee that every state-action pair gets visited, but they come with practical problems that make them hard to use in real-world settings.

### Question 1

*What does the term Monte Carlo refer to when discussing algorithms for Reinforcement Learning?*

In reinforcement learning, **Monte Carlo** refers to methods that learn value functions by averaging the returns from complete episodes of experience (Sutton and Barto 2018). The name comes from the Monte Carlo casino in Monaco, which makes sense given that these methods rely on repeated random sampling—the agent does a lot of random things and learns from the outcomes.

Here is how it works: an agent runs through an episode by interacting with the environment, collects the rewards along the way, and then computes the return  $G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$  for each state or state-action pair it visited. After many episodes, the agent averages those returns to get its value estimates. What makes Monte Carlo methods stand apart from other RL approaches is that they are **model-free**—the agent does not need to know the transition probabilities  $p(s', r | s, a)$ —and they do **not bootstrap**, meaning updates use actual complete returns rather than estimates of what the next state is worth. The downside is that the agent has to wait until the episode ends before it can learn anything from it.

### Question 2

*Write down the famous Bellman equation in Reinforcement Learning. Discuss it briefly.*

Copyright © 2024, Association for the Advancement of Artificial Intelligence (www.aaai.org). All rights reserved.

The **Bellman equation** for the state-value function under a policy  $\pi$  is (Sutton and Barto 2018):

$$v_{\pi}(s) = \sum_a \pi(a | s) \sum_{s', r} p(s', r | s, a) [r + \gamma v_{\pi}(s')]$$

What this equation says is that the value of being in state  $s$  under policy  $\pi$  is the expected immediate reward plus the discounted value of whatever state the agent ends up in next. It averages over all actions the policy might take (weighted by  $\pi(a | s)$ ) and all possible transitions (weighted by the dynamics  $p(s', r | s, a)$ ).

This equation is important because it breaks the value function into two pieces: the reward the agent gets right now and the value of where it goes next. That breakdown is what makes iterative solution methods possible. Dynamic Programming algorithms like policy evaluation apply this equation over and over as an update rule until the values converge. Monte Carlo and Temporal Difference methods do the same thing, but with samples instead of the full model, which is useful when the agent does not know the environment’s dynamics.

### Question 3

*Some naïve algorithms for learning  $V(s)$  and  $Q(s, a)$  values are called Exploring Start algorithms.*

**(a) What does the term Exploring Start mean? How do such algorithms work?**

**Exploring Starts** means that every episode begins with a randomly chosen state-action pair  $(S_0, A_0)$ , where every possible pair has some chance of being picked as the starting point (Sutton and Barto 2018). The idea is that if you run enough episodes, every state-action pair will eventually be visited as the first step of an episode.

For learning  $V(s)$ , the **First-Visit MC Prediction** algorithm generates an episode starting from a random state and follows policy  $\pi$ . It then works backward from the end of the episode, computing the return  $G_t$  at each step. For the first time each state  $S_t$  appears in the episode, the algorithm adds  $G_t$  to a running list and updates  $V(S_t)$  as the average of all the returns it has seen for that state.

For learning  $Q(s, a)$ , the **Monte Carlo ES** algorithm does the same thing but for state-action pairs. It randomly picks

an initial  $(S_0, A_0)$  as the exploring start, runs the episode following  $\pi$  from there, computes returns backward, and updates  $Q(S_t, A_t)$  as the average return for each first-visited pair. Then it improves the policy by going greedy:  $\pi(s) \leftarrow \arg \max_a Q(s, a)$ . This cycle of evaluation and improvement repeats until the policy converges to  $\pi_*$ .

### (b) Why is exploration necessary in Reinforcement Learning?

Without exploration, the agent can get stuck with a **suboptimal policy**. If the agent always picks the best action it currently knows about (pure greedy), it will never try other actions that might actually be better. Say the agent is in a state where action  $a_1$  gives a moderate reward and action  $a_2$  has never been tried—without exploration, the agent will never find out that  $a_2$  might lead to a higher return. This is the **exploration-exploitation dilemma**: the agent has to balance doing what it thinks is best right now (exploitation) with trying new things to improve what it knows (exploration) (Sutton and Barto 2018). The agent needs enough exploration so that all state-action pairs get sampled enough times for the Q-value estimates to converge to their true values.

### (c) What are some issues with Exploring Start algorithms?

The biggest problem with Exploring Starts is that the starting assumption is **impractical in most real-world environments**. A self-driving car cannot teleport to an arbitrary state, and a robot cannot just be placed in any configuration at will. Most real environments have fixed starting conditions, so the idea that every episode can start from any random state-action pair is not realistic.

There is also a **scalability problem**. In environments with large or continuous state-action spaces, the chance of randomly picking any particular pair as the start becomes extremely small. To get adequate coverage of all pairs, the algorithm would need a huge number of episodes, which may not be practical.

Finally, Exploring Starts is **not usable for online or continuing tasks** where the agent cannot reset the environment to an arbitrary starting point. These limitations are why on-policy alternatives like  $\epsilon$ -greedy methods exist—they handle exploration *during* the episode instead of relying on the starting conditions (Sutton and Barto 2018).

## AI Use Statement

Claude (Anthropic) was used to format my answers in AAAI LaTeX format.

## References

Sutton, R. S.; and Barto, A. G. 2018. *Reinforcement Learning: An Introduction*. Cambridge, MA: MIT Press, 2nd edition. Available free at <http://incompleteideas.net/book/the-book-2nd.html>.